

InsightSent: Explainable Sentiment Analysis Using Logistic Regression, Naive Bayes and LightGBM

Ansu Nanda

Heritage Institute of Technology
Computer Science and Engineering

Mentor Name: Arpan Roy

Period of Internship: 18th May 2026 – 31st July 2026

Report submitted to: IDEAS – Institute of Data
Engineering, Analytics and Science Foundation, ISI
Kolkata

1. Abstract

Sentiment analysis is an essential component to identify and analyze the sentiments and feedback created by customers or viewers in an online medium. The current project discusses InsightSent, an explainable multi-model system which is designed for sentiment analysis with traditional machine learning methods. This system involves intensive preprocessing of the text with the inclusion of negation support, stemming, removal of stop-words and TF-IDF extraction with the help of uni-grams and bi-grams. The three machine learning models such as Logistic Regression, Multinomial Naive Bayes and LightGBM are trained on the IMDB Movie Review Dataset which is collected from Kaggle. The performance of the three models is assessed with the use of accuracy, precision, recall, F1-score, and Precision-Recall curves. In addition to the standard process of building sentiment classifiers, the current project also includes model-based explanations to emphasize the words having positive and negative impacts on each of the predictions. The trained models are then embedded into an interactive prediction dashboard.

2. Introduction

The uprising of online platforms has led to a large number of reviews and feedback provided by the users. Automatic processing of textual data containing the user's opinion would help organizations to make rational decisions regarding their products and services and understand the degree of their popularity among customers. Sentiment analysis, one of the most common applications of NLP which implies the classification of textual information as positive or negative.

Classical machine learning methods prove to be quite efficient for sentiment analysis because of their high performance, simplicity and interpretability. For this project, an explainable sentiment analysis system, called InsightSent, was created using TF-IDF features and three different supervised machine learning classifiers such as Logistic Regression, Multinomial Naive Bayes and LightGBM. Text pre-processing consists of normalization, custom stop-words elimination, handling of negations, stemming and TF-IDF vectorization based on unigrams and bigrams to create informative textual representations. All three models will be analyzed in terms of their performance based on classification metrics and Precision-Recall diagrams.

In addition to accurate predictions, one of the key goals of this project is to explain how the predictions have been made. That is why an explainability part of the system has been introduced which would help to detect the informative words responsible for a certain prediction of each model.

According to the internship schedule, the following topics were covered during the initial two weeks:

1. Basic Statistics for Data Science
2. Data Visualization
3. Introduction to GitHub and Cloud Computing

4. Data Science Application Development using Streamlit
5. Machine Learning – I
6. Machine Learning – II
7. LLM Fundamentals
8. Communication Skills
9. Deep Learning – I

This project was developed during the internship by applying concepts learned in statistics, machine learning, deep learning, data visualization and github.

3. Project Objective

- To build an intelligent sentiment analysis system that automatically classifies movie reviews into positive and negative classes.
- To build an effective text pre-processing system that involves text cleaning, negation handling, custom stop words removal, stemming and finally TF-IDF features extraction in unigram and bigram forms
- To build and compare the performance of different machine learning algorithms: Logistic Regression, Multinomial Naive Bayes and LightGBM, using common evaluation techniques of classification.
- To improve the transparency of sentiment prediction by building an explainability system that finds the most impactful words for the prediction made by each model.
- To deploy the models built using an interactive user interface for the real-time prediction of sentiment with confidence scores along with the explainability information provided by the corresponding model.

4. Methodology

4.1 Tools and Technologies used

Category	Tool / Library	Purpose
Environment	Kaggle Notebooks	Cloud-based execution

Language	Python 3	Core implementation
Data handling	Pandas, NumPy	Data loading, manipulation
Visualization	Matplotlib, Seaborn, WordCloud	EDA and result plotting
NLP	NLTK (punkt, stopwords), re	Tokenization, preprocessing
Feature extraction	Scikit-learn TfidfVectorizer	Text-to-numeric conversion
ML models	Scikit-learn (Logistic Regression, Naive Bayes), LightGBM	Sentiment classification
Evaluation	Scikit-learn metrics	Accuracy, Precision, Recall, F1 score
Model persistence	Joblib	Saving and loading trained models
Interface	ipywidgets, IPython.display	Interactive prediction widget

4.2 Dataset

The data set that has been utilized in this study is the IMDB Dataset of 50,000 movie reviews that is freely available on Kaggle through this link:

[kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews](https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews)

It has been imported straight to the Kaggle notebook environment through the Kaggle Datasets API service. It is made up of two columns, first is review (movie review raw text) and the second is sentiment (positive or negative sentiment). There are exactly 24,884 positive and 24,698 negative sentiments contained within the data set, thus making it well-balanced for binary classification problems.

No survey has been conducted for this study as all data has been taken from this existing publicly available data set.

4.3 Data Exploration and Visualisation

There was an exploratory data analysis (EDA) process performed to gain insight about the nature and distribution of the data.

Step 1: Removal of duplicates- There was a check to detect duplicate records in the dataset using `df.drop_duplicates(inplace=True)` to make sure that none of the reviews was duplicated in the training set.

Step 2: Class distribution analysis- Pie charts were created to validate that the dataset is balanced where there are 50.2% of positive and 49.8% of negative reviews. This difference is negligible.

Step 3: Text Preprocessing- The raw movie reviews have high levels of noise, in the form of HTML tags, URLs, punctuations, common function words, and the same root words that may vary in morphology. A multi-step preprocessing pipeline was designed and applied uniformly to both training data and inference-time input

Step 4: Frequency of words- The top 10 frequent words were obtained and shown as horizontal bar charts in two subplots before and after preprocessing. This was done in order to make sure that the process of preprocessing was properly removing the noise and keeping the content words.

Step 5: Creating word clouds- The creation of word clouds for positive and negative reviews was performed to intuitively see which words are dominant in each sentiment class.

4.4 Text Preprocessing Pipeline

The raw movie reviews have high levels of noise, in the form of HTML tags, URLs, punctuations, common function words, and the same root words that may vary in morphology. A multi-step preprocessing pipeline was designed and applied uniformly to both training data and inference-time input

The preprocessing pipeline consisted of the following sequential steps:

Step 1: Lowercasing- All text was converted to lowercase to ensure case insensitive matching so that for example "Good" and "good" are treated identically.

Step 2: HTML tag removal- IMDB reviews frequently contain embedded
 tags from the original web interface. These were removed using `re.sub('<br />', '', text)`.

Step 3: URL removal- Any URLs present were removed using the pattern `re.sub(r'http\S+|www\S+', '', text)`.

Step 4: Punctuation removal- All non-alphanumeric, non-whitespace characters were removed using `re.sub(r'[^\w\s]', '', text)` that reduced the token space to words and numbers only.

Step 5: Tokenization- The cleaned text was split into individual word tokens using NLTK's `word_tokenize()` function.

Step 6: Negation tagging- A custom rule-based negation tagging function `apply_negation_tagging()` was applied before stopwords removal. This step addresses a well-known limitation of bag-of-words models: without special handling, a phrase like "not good" gets split into two separate tokens not and good, where good dominates and misleads the model toward a positive prediction. The negation tagger converts the word immediately following a negation cue (not, no, never, nor, neither, n't) into a fused token with a not_ prefix, for example, "not good" becomes ["not", "not_good"]. This encodes the negation context directly into the token, making it visible to a grammar-blind bag-of-words model. This approach is consistent with the NOT_ prefix method which is documented in existing sentiment analysis literature (IOP Conference Series, 2018; Lex2Sent, arXiv:2209.13023, 2022).

Step 7: Stopword removal with negation protection- The stopwords of standard English were imported from the NLTK corpus, and six words that express negations like 'not', 'no', 'never', 'nor', 'neither', and 'n't' were explicitly excluded from the stopwords before applying the filter. This approach ensured that negations were retained in the stream of tokens while common function words such as 'the', 'a', 'is', 'was' were filtered out.

Step 8: Porter Stemming- All other tokens were stemmed to their root form using the NLTK PorterStemmer function. This maps morphological variants to a common stem, for example, "running", "runs", and "ran" all map to "run", and "movie" maps to "movi". This reduces vocabulary size.

The processed text is saved in a new column called `cleaned_review`, and `preprocessedText()` function was used uniformly throughout the training and inference process.

4.5 Feature Extraction

Machine learning models cannot directly process raw text. The preprocessed reviews were transformed into feature vectors through TF-IDF vectorization using `TfidfVectorizer` of Scikit-learn library.

The vectorizer was configured with the following parameters:

- `max_features=7000`: It keeps the 7,000 most significant terms of the vocabulary.

- `ngram_range=(1, 2)`: It allows to take into account unigrams and bigrams and it lets the models learn the context of two-word sequences such as `not_good` along with unigrams

The vectorizer was trained solely on the training dataset. As a result, the feature matrices had dimensions $(39,665 \times 7,000)$ for training and $(9,917 \times 7,000)$ for testing. The sentiment labels were represented as binary integers where positive: 1 and negative: 0.

4.6 Train-Test Split

The dataset was split into training and testing sets using an 80:20 ratio via Scikit-learn's `train_test_split` function with `random_state=42` for reproducibility.

- Training set: 39,665 samples (80%)
- Test set: 9,917 samples (20%)

The `random_state=42` parameter ensures that the exact same split is reproduced on every run, making results comparable and reproducible.

4.7 Model Selection

1. Logistic Regression: It is selected as the linear baseline. It learns one weight per TF-IDF feature, making it interpretable and computationally efficient. This model suits high-dimensional sparse text feature matrices quite well.

2. Multinomial Naive Bayes: It is selected as the probabilistic baseline. It learns the likelihood of each word given each class using Laplace smoothed conditional probabilities and it is known for fast training and robust performance on text data despite the naive conditional independence assumption.

3. LightGBM (Light Gradient Boosting Machine): This model is an ensemble tree-based model. The algorithm builds an additive ensemble of decision trees sequentially, where every tree corrects the errors of the previous tree. LightGBM's histogram-based splitting and leaf-wise tree growth make it faster than traditional gradient boosting algorithms while maintaining competitive accuracy.

4.8 Model Evaluation

Each trained model was evaluated on the test set (`X_test`) using the following metrics:

- **Accuracy:** proportion of correctly classified reviews
- **Precision:** Out of all predicted positives, what fraction were true positive
- **Recall:** Out of all truly positive reviews, what fraction were correctly identified
- **F1 Score:** harmonic mean of Precision and Recall.
- **Confusion Matrix:** visualized using `ConfusionMatrixDisplay` to show True Positive, True Negative, False Positive, and False Negative counts
- **Precision-Recall Curve:** plotted for all three models simultaneously with Average Precision (AP) scores to assess performance across different classification thresholds

The result DataFrame has been created by storing the metrics of each model in a dictionary and converting it to a table form. A bar graph has been created to compare the metrics of all the three models.

4.9 Explainable Inference Interface

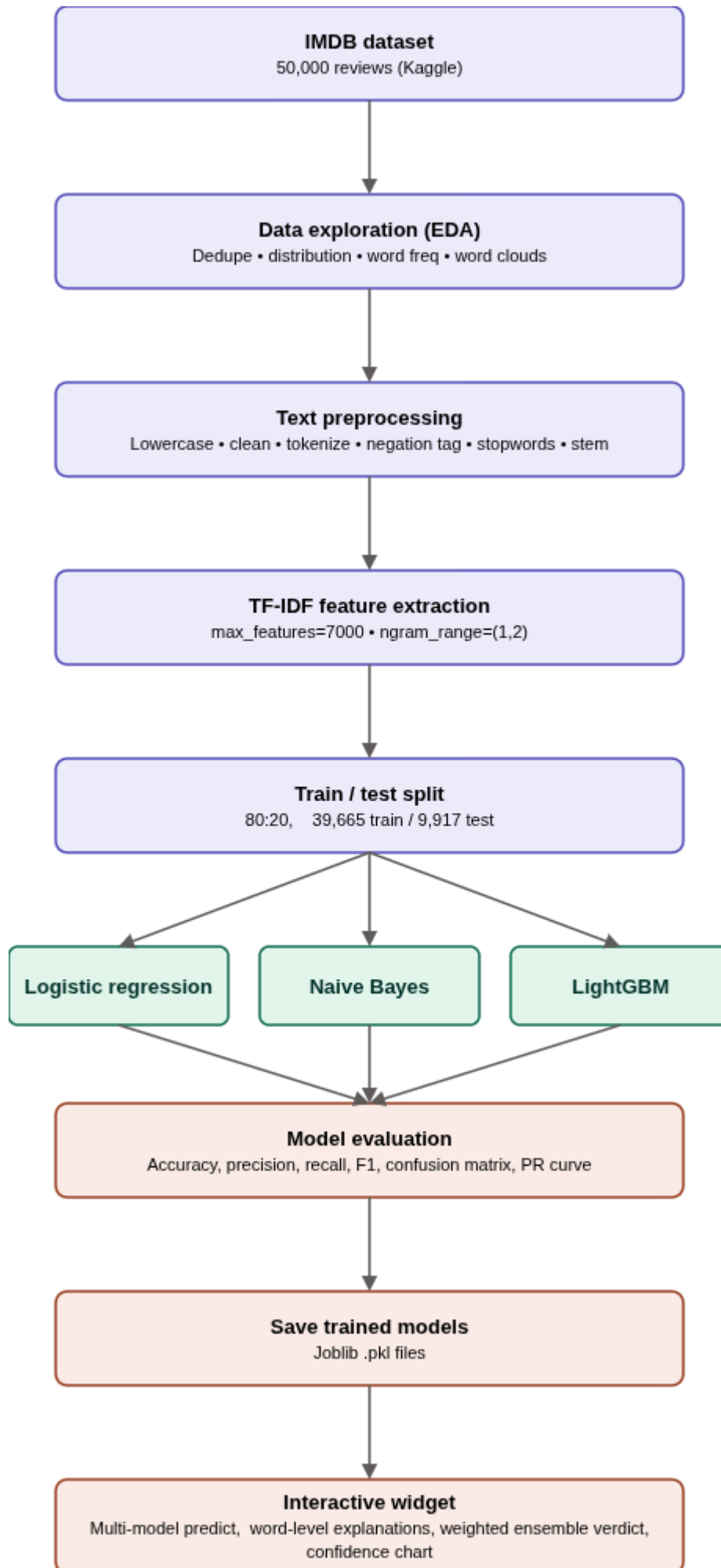
An interactive prediction interface was developed using ipywidgets within the Kaggle notebook environment. The interface accepts a movie review from the user and returns predictions from all three models simultaneously.

The explainability module calculates word-level contribution score for each of the models separately using model specific techniques:

- **Logistic Regression:** Contribution of each word = TF-IDF weight $\times$ learned coefficient. Words with positive coefficients push the prediction toward positive and negative coefficients push the prediction toward negative.
- **Naive Bayes:** Contribution = TF-IDF weight $\times$ log-odds ratio, where the log-odds ratio is $\log P(\text{word}|\text{positive}) - \log P(\text{word}|\text{negative})$. A positive number means that the word appears more often in positive reviews.
- **LightGBM:** Contributions computed using TreeSHAP by using the parameter `pred_contrib=True` which goes through each sample through every tree in the ensemble and measures marginal contribution of each feature to the final prediction score.

The top 3 most influential words for the positive sentiment and the top 3 most influential words for the negative sentiment are derived and presented according to each of the models. The ensemble decision for sentiment is calculated by summing up confidence scores of positive predictions and negative predictions for all three models.

4.10 Methodology Flowchart



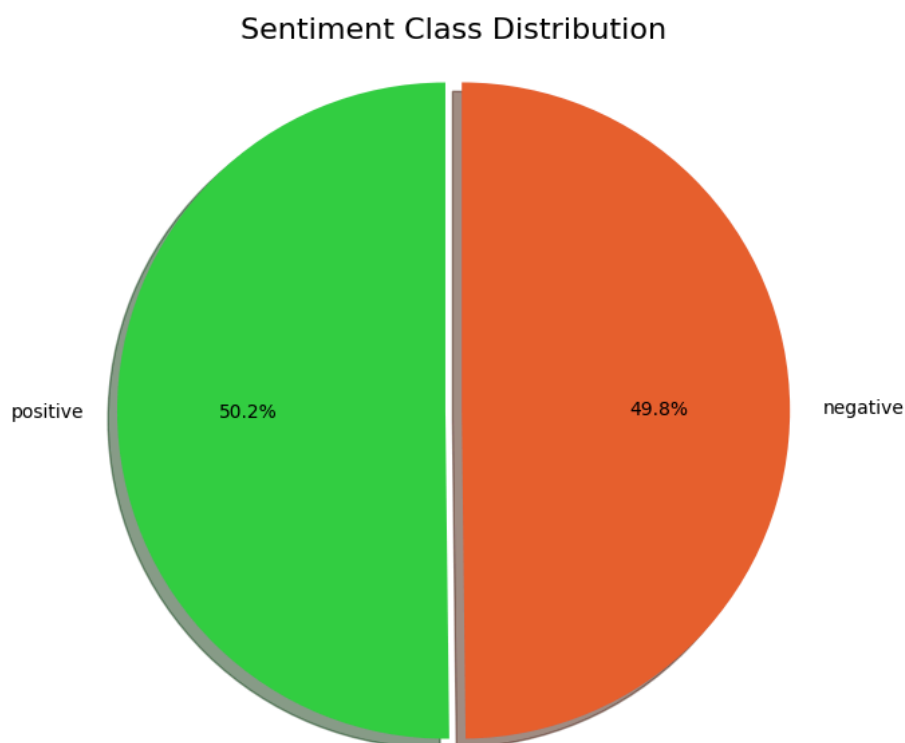
4.11 Source Code

The complete implementation is available on GitHub:

<https://github.com/Ansu-01/InsightSent-Explainable-Sentiment-Analysis-Using-Logistic-Regression-Naive-Bayes-and-LightGBM/tree/main>

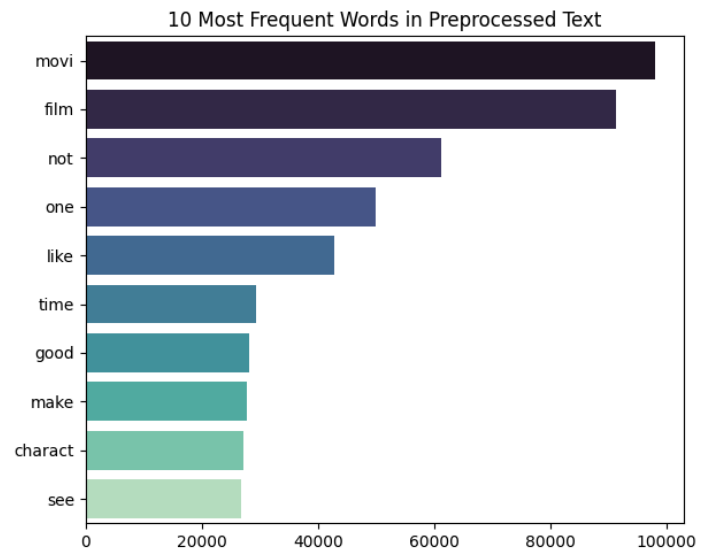
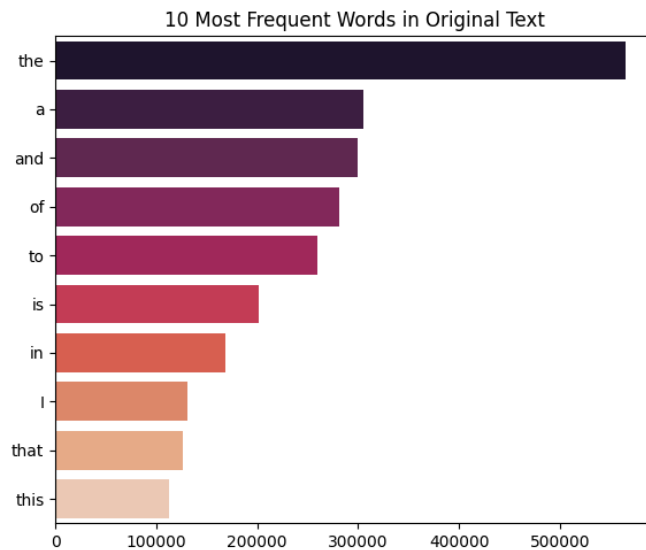
5. Data Analysis and Results

5.1 Sentiment Class Distribution

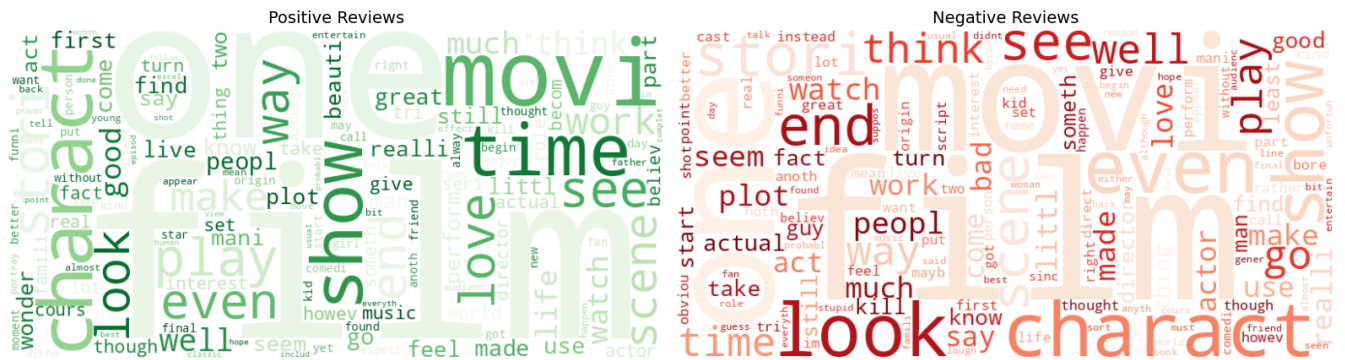


5.2 Word Frequency Analysis Before and After Preprocessing

Below, the left chart shows the 10 most frequent words in the original text, dominated entirely by function words such as 'the', 'a', 'and' and 'of' which carry no sentiment signal. The right chart shows after preprocessing where these have been eliminated and the vocabulary is now led by content words such as 'movi', 'film', 'not', 'one' and 'like' and they carry semantic meaning. The presence of 'not' in the top 10 preprocessed words confirms that the negation words protection step is working correctly.



5.3 Word Cloud Analysis



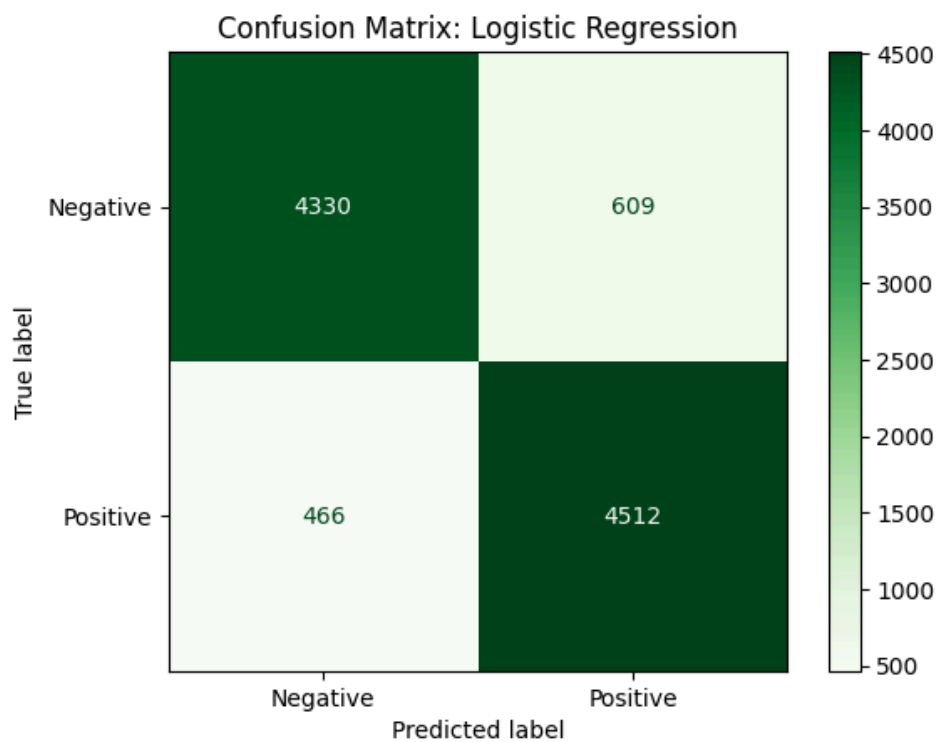
5.4 Vocabulary Analysis

Property	Value
Total unique tokens (unigrams + bigrams)	3045060
Features retained	7,000
N-gram range	(1, 2): unigrams and bigrams
Vocabulary Coverage	$7,000 / 3,045,060 \times 100 = 0.23\%$

5.5 Model Results

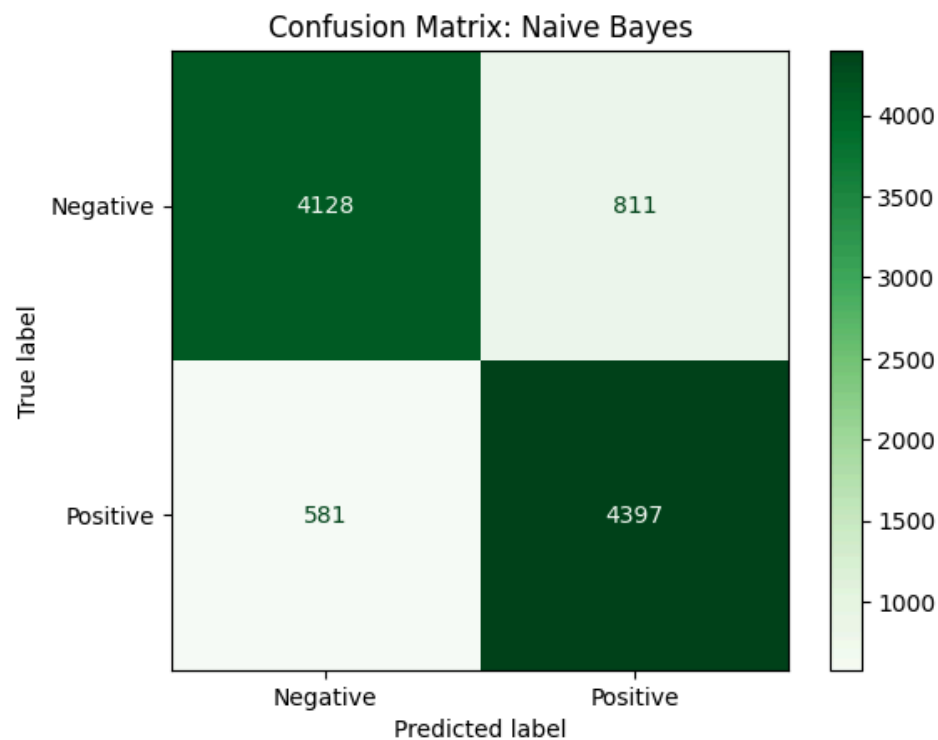
5.5.1 Logistic Regression

Classification Report of Logistic regression:					
		precision	recall	f1-score	support
	0	0.90	0.88	0.89	4939
	1	0.88	0.91	0.89	4978
accuracy				0.89	9917
macro avg		0.89	0.89	0.89	9917
weighted avg		0.89	0.89	0.89	9917



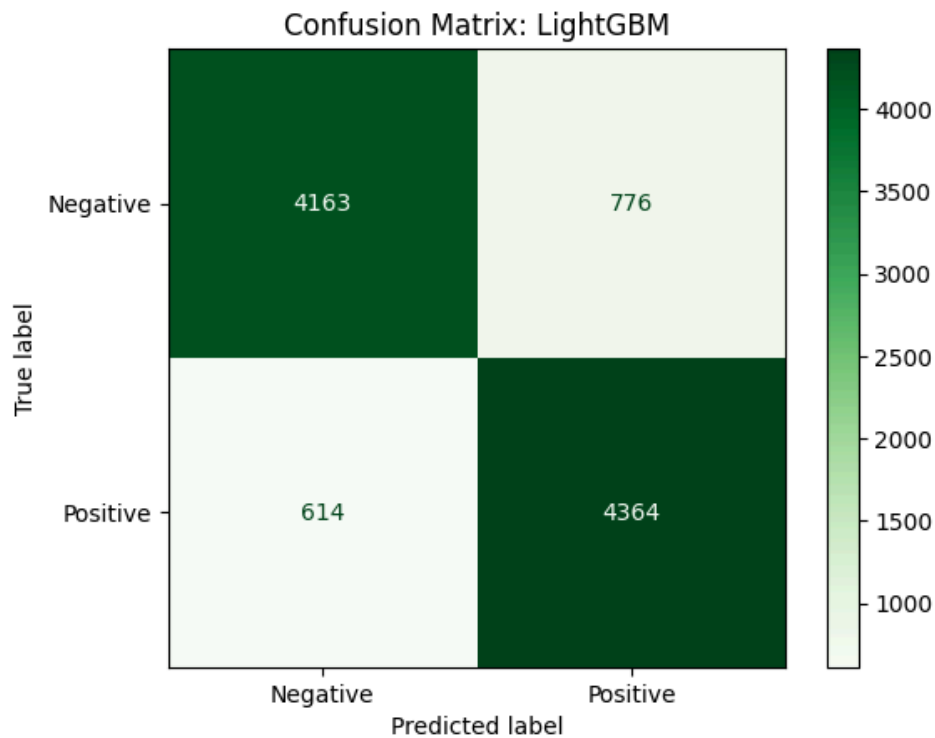
5.5.2 Naive Bayes

Classification Report of Naive Bayes:					
	precision	recall	f1-score	support	
0	0.88	0.84	0.86	4939	
1	0.84	0.88	0.86	4978	
accuracy			0.86	9917	
macro avg	0.86	0.86	0.86	9917	
weighted avg	0.86	0.86	0.86	9917	



5.5.3 LightGBM

Classification Report of LightGBM:					
	precision	recall	f1-score	support	
0	0.87	0.84	0.86	4939	
1	0.85	0.88	0.86	4978	
accuracy			0.86	9917	
macro avg	0.86	0.86	0.86	9917	
weighted avg	0.86	0.86	0.86	9917	



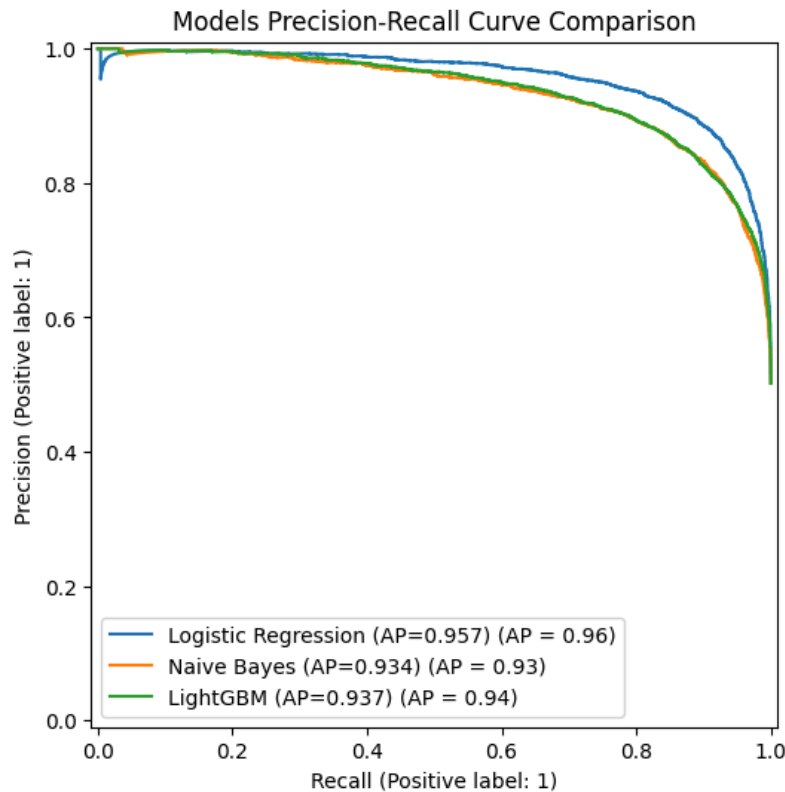
5.6 Comparative Analysis of All Three Models

Logistic Regression outperformed both Naive Bayes and LightGBM across all four evaluation metrics.

	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.891600	0.881078	0.906388	0.893554
Naive Bayes	0.859635	0.844278	0.883286	0.863342
LightGBM	0.859837	0.849027	0.876657	0.862621

5.7 Precision-Recall Curve Comparison

Average precision is highest in Logistic Regression with 0.957, followed by LightGBM with 0.937 and then Naïve Bayes with 0.934. These three models have their curves close to the top left corner throughout the range of recall values, indicating that high precision is maintained even as recall increases.



6. Conclusion

This project successfully developed InsightSent, an explainable multi-model sentiment analysis system for sentiment analysis, which is able to classify IMDB movie reviews as either positive or negative accurately. This proves that classical machine learning algorithms along with proper preprocessing and feature extraction, can perform effectively on an NLP benchmark dataset.

Among all the three models that have been examined, Logistic Regression performed the best in terms of all metrics, including Accuracy of 89.16%, Precision of 88.11%, Recall of 90.64%, and F1-Score of 89.36%, with an Average Precision of 0.957 on the Precision-Recall curve. Therefore, it is established that a linear model can be a very good baseline for high-dimensional and sparse TF-IDF features. Multinomial Naïve Bayes and LightGBM both performed equally well, achieving an accuracy of 85.96% and 85.98% respectively, while having F1-Score of 86.33% and 86.26%.

An important key finding of this project was the critical importance of negation handling in text preprocessing. The bag of words methodology considers negation words such as "not" to be stopwords, thereby misrepresenting sentences such as "not good" in terms of "good." This problem was overcome by incorporating a specialized negation tagger that combines negations with the next word, for example, "not good" becomes "not_good" and thereby

maintains the semantic inversion in the feature space itself. Along with this fix, a bigram TF-IDF vectorization method (`ngram_range = (1,2)`) proved to be more effective.

Another key finding is that even though there were 3,045,060 tokens unique for the unigrams and bigrams, the selection of only 7,000 features would be enough to obtain almost optimal classification results since most of the sentiment information is embedded in the high-frequency discriminatory words.

The explainability module added significant value beyond standard classification process. By computing word level contribution scores using model-specific techniques such as coefficient weights for Logistic Regression, log odds ratios for Naive Bayes, and TreeSHAP like contributions through `pred_contrib=True` for LightGBM. It allows for transparent reasoning for all the predictions made by identifying the top contributing words for each prediction.

7. Future Works

- Transformer-based models such as BERT or DistilBERT can be explored to capture deeper contextual and long-range dependencies that are not inherently possible for TF-IDF to capture which can help in achieving accuracy above the 92-94% range.
- Multilingual extension using multilingual TF-IDF or cross-lingual embeddings can broaden the system's applicability beyond English language based reviews.
- An aspect-level sentiment analysis will help to expand the system from the document-level classification to the level where one can identify which particular aspects such as acting, direction etc. of the movie have been marked with positive/negative sentiments.

8. APPENDICES

1. <https://arxiv.org/abs/2209.13023>
2. [Github link](#)
3.  project report